

BERT-Embedded Self-Attention Transformer Recommender: Tackling Sparsity and Cold-Starts in Large-Scale Recommender Systems.

Abstract

Recommender systems are critical infrastructure for modern digital platforms, yet they face persistent challenges with cold-start scenarios and extreme data sparsity. These issues significantly degrade recommendation quality and user experience, particularly affecting new users and items with limited interaction history. Despite advances in deep learning, existing approaches struggle to effectively integrate heterogeneous information sources, limiting their ability to address sparsity-induced prediction errors and overfitting. This research develops a unified recommender architecture that jointly leverages textual semantics, structured metadata, and collaborative signals to mitigate cold-start and sparsity challenges while achieving superior rating prediction accuracy across diverse domains and interaction densities. We propose BEST-Rec, a BERT-Embedded Self-Attention Transformer Recommender that represents a novel hybrid architecture featuring: (1) dual complementary embeddings combining BERT-based semantic vectors for textual attributes and low-rank SVD embeddings for collaborative signals; (2) a bidirectional cross-attention mechanism enabling fine-grained user-item preference alignment during feature learning; and (3) a composite training objective integrating smooth L1 regression loss with cross-entropy classification for robust optimization.

We evaluate BEST-Rec on Amazon Beauty and Books datasets characterized by extreme sparsity with cold entities ranging from 75% to 98%. Performance is assessed through 5-fold cross-validation using MAE and RMSE metrics against state-of-the-art baselines including matrix factorization, deep learning methods, and recent transformer-based approaches. BEST-Rec demonstrates superior performance across both domains. On the Beauty dataset, our model achieves 36.2% MAE reduction and 18.2% RMSE improvement compared to average baseline performance. On the Books dataset, BEST-Rec attains 28.8% MAE reduction and 10.8% RMSE improvement over baseline averages. Comprehensive ablation studies confirm the critical importance of cross-attention fusion and dual-source embeddings, with textual data proving essential for substantial performance gains in sparse scenarios. By enabling dynamic interaction between textual semantics, structured metadata, and collaborative signals, BEST-Rec provides a principled solution to sparsity and cold-start challenges, offering a robust foundation for large-scale recommendation systems.

1. Introduction

Recommender systems have evolved into critical infrastructure components that process vast volumes of user-item interactions daily, representing one of the fastest-growing sectors in artificial intelligence applications. These sophisticated information-filtering mechanisms predict user preferences across diverse domains—from e-commerce platforms serving extensive product catalogs to streaming services managing massive content libraries—directly influencing user engagement and business performance. Leading platforms consistently report substantial improvements in conversion rates, user session duration, and overall user satisfaction through effective recommendation algorithms (Najafabadi et al., 2025; Nguyen et al., 2024; Chen et al., 2020). However, despite their commercial success, modern recommender systems face escalating technical challenges as data volumes, user expectations, and application complexity continue to grow exponentially.

Recommender systems research has historically evolved through three foundational paradigms, each addressing distinct technical challenges through specialized algorithmic approaches. Content-Based Filtering (CBF) leverages item feature vectors and user preference profiles to identify similarity patterns, employing techniques ranging from TF-IDF vectorization to deep neural embeddings for feature extraction (Hasan et al., 2024). While CBF demonstrates robustness in handling new user scenarios through rich metadata exploitation, it fundamentally suffers from feature representation limitations and tends toward over-specialization, constraining recommendation diversity and serendipitous discovery. Collaborative Filtering (CF) operates on the principle of collective intelligence, utilizing matrix factorization techniques, neighborhood-based methods, or latent factor models to identify user-user or item-item similarities from interaction patterns without explicit content analysis (Najafabadi et al., 2017; Koren et al., 2009). However, CF approaches encounter severe performance degradation under data sparsity conditions, with typical user-item matrices exhibiting sparsity levels, and face fundamental cold-start limitations when insufficient interaction history exists. Contemporary hybrid methodologies have evolved beyond simple linear combinations toward sophisticated deep learning architectures, incorporating graph neural networks, attention mechanisms, and multi-modal fusion strategies to synergistically combine content and collaborative signals (Wang et al., 2022; Najafabadi et al., 2019; Chen et al., 2020), yet these approaches still struggle with effective cross-modal information integration in extremely sparse environments.

Deep learning has fundamentally transformed recommender systems through neural collaborative filtering, graph neural networks for complex relationship modeling, and transformer architectures for sequential pattern recognition (He et al., 2017; Liu et al., 2021; Kim et al., 2024). Recent advances include BERT-based review analysis and large language model integration for enhanced representation learning (Wu et al., 2024). However, current approaches predominantly process heterogeneous data modalities—textual reviews, numerical metadata, and collaborative signals—through independent pipelines or simple concatenation, failing to capture essential cross-modal interactions for robust performance in sparse environments.

Despite these advances, recommender systems continue to face fundamental scalability and accuracy challenges that have become more pronounced with increasing data volumes and user expectations. The cold-start problem remains critical, particularly affecting platforms with high user and item turnover rates, where new users and items struggle to receive meaningful recommendations due to insufficient interaction data (Li et al., 2023). Data sparsity compounds these challenges: typical user-item interaction matrices exhibit extreme sparsity levels, making reliable pattern discovery challenging and driving models toward overfitting observed interactions while failing to generalize effectively (Hasan et al., 2024; Najafabadi et al., 2017). Current approaches exhibit a fundamental architectural limitation: they process heterogeneous data modalities—unstructured review texts, structured metadata, and latent collaborative factors—independently or through simple concatenation strategies, preventing the capture of complex cross-modal interactions essential for robust recommendations (Wang et al., 2022; Kharidia et al., 2024). Recent transformer-based approaches, while showing promise for sequential recommendation and review analysis (Kim et al., 2024; Shuai et al., 2022), lack unified architectures capable of jointly modeling textual semantics, numerical features, and collaborative signals within a cohesive attention framework. This limitation becomes particularly acute in sparse scenarios where leveraging all available information sources is crucial for maintaining recommendation quality.

To address these fundamental limitations, we propose BEST-Rec (BERT-Embedded Self-Attention Transformer Recommender), a novel hybrid architecture that unifies diverse information sources through a sophisticated bidirectional attention mechanism. BEST-Rec integrates three complementary

components: transformer-based encoders leveraging pre-trained language models to capture semantic dependencies in user and item review texts (Kim et al., 2024; Liu et al., 2023), dense embeddings of structured item metadata including pricing and rating information, and collaborative factors derived from low-rank matrix factorization to inject global preference signals (Koren et al., 2009). A bidirectional cross-attention module dynamically aligns user-text embeddings with item representations, enabling fine-grained preference-item matching that adapts to individual user contexts, addressing the limitations identified in recent survey work (Hasan et al., 2024; Najafabadi et al., 2025).

Our contributions are fourfold:

1. We introduce a unified transformer-based embedding architecture that jointly processes user reviews, item reviews, structured metadata, and collaborative latent factors within a single end-to-end framework
2. We develop a bidirectional cross-attention fusion mechanism that enables fine-grained alignment between user and item textual representations during feature learning.
3. We conduct extensive experiments on large-scale Amazon Books and Beauty datasets, demonstrating state-of-the-art performance improvements with 36.2% MAE reduction and 18.2% RMSE improvement compared to established baselines.
4. We provide comprehensive analysis of model behavior across various scenarios including extreme sparsity and cold-start conditions, validating robustness and generalizability.

The remainder of this paper is structured as follows: Section 2 surveys prior content-based, collaborative, and hybrid methods, with particular focus on transformer models and cross-modal fusion strategies relevant to recommendation systems. Section 3 details our methodology, including data preprocessing pipelines, embedding strategies, and the complete BEST-Rec architecture. Section 4 describes experimental setups, datasets, and evaluation protocols ensuring reproducibility. Section 5 presents comprehensive results including main performance comparisons, ablation studies, and convergence analysis. Section 6 discusses findings, implications, and practical considerations for deployment. Section 7 addresses current limitations and outlines promising directions for future research. Section 8 concludes with a summary of contributions and their broader impact on the recommender systems field.

2. Literature Review

Recommender systems have undergone substantial evolution to address fundamental challenges that persist across domains and applications. Two critical challenges continue to impede recommendation quality: cold-start problems, arising from insufficient user-item interaction data for new users or items, and data sparsity, characterized by extremely sparse user-item interaction matrices that hinder reliable pattern detection and risk model overfitting. These challenges have driven the development of increasingly sophisticated methodologies that integrate diverse information sources and modeling approaches (Najafabadi et al., 2025; Hasan et al., 2024).

The evolution of recommender systems can be characterized by several paradigmatic shifts: from simple content-based and collaborative filtering approaches to sophisticated deep learning architectures, from homogeneous to heterogeneous graph modeling, and from unimodal to multimodal information integration. Each advancement has contributed to addressing specific limitations while introducing new challenges, particularly regarding computational complexity and scalability.

2.1 Traditional Recommendation Paradigms

2.1.1 Content-Based Filtering Approaches

Content-Based Filtering (CBF) represents one of the earliest systematic approaches to recommendation, leveraging explicit item attributes and user profiles to suggest items with similar characteristics to those previously enjoyed by users. CBF naturally addresses item cold-start scenarios when sufficient descriptive metadata is available, enabling meaningful recommendations for newly introduced items without requiring interaction history (Najafabadi et al., 2017).

The strength of CBF lies in its interpretability and robustness to data sparsity in the collaborative dimension. By focusing on item features such as genre, category, or textual descriptions, CBF can provide reasonable recommendations even when user-item interactions are extremely sparse. However, CBF suffers from several fundamental limitations that restrict its applicability in modern recommendation scenarios.

Over-specialization represents a critical weakness, where recommendations become increasingly narrow and predictable, limiting users' exposure to diverse content beyond their established preferences. This phenomenon, often termed the "filter bubble" effect, reduces serendipitous discovery and can lead to user satisfaction decline over time (Lops et al., 2011). Additionally, CBF struggles significantly with user cold-start scenarios, as new users without interaction histories cannot be accurately profiled, limiting the system's ability to provide personalized recommendations.

Recent advances in CBF have incorporated sophisticated text processing techniques, including word embeddings (Word2Vec, GloVe) and transformer-based encodings (BERT, RoBERTa), to better capture semantic relationships between items (Noorian et al., 2024; Najafabadi et al., 2025). However, these improvements primarily address representation quality rather than the fundamental limitations of content-only approaches.

2.1.2 Collaborative Filtering Evolution

Collaborative Filtering (CF) emerged as a powerful alternative that leverages collective user behavior patterns to generate recommendations without requiring explicit item attributes. The underlying assumption—that users with similar interaction patterns will exhibit similar preferences—has proven remarkably effective across diverse domains (He and McAuley, 2016).

Memory-based CF methods, including user-based and item-based collaborative filtering, operate on similarity computation between users or items based on interaction patterns. While intuitive and interpretable, these approaches face significant scalability challenges as user and item populations grow. The computational complexity of pairwise similarity calculations becomes prohibitive in large-scale scenarios, limiting their practical applicability.

Model-based CF approaches, particularly matrix factorization techniques, have dominated collaborative filtering research due to their superior scalability and performance characteristics. Singular Value Decomposition (SVD), Non-negative Matrix Factorization (NMF), and Probabilistic Matrix Factorization (PMF) effectively capture latent factors representing user preferences and item characteristics in lower-dimensional spaces (Koren et al., 2009).

Advanced matrix factorization variants have incorporated additional signals to enhance performance. SVD++ integrates implicit feedback alongside explicit ratings, while timeSVD++ captures temporal dynamics in user preferences. Factorization Machines (FM) extend traditional matrix factorization to handle arbitrary feature interactions, enabling integration of additional contextual information (Zhang, 2022).

Despite these advances, CF methods face persistent limitations: severe performance degradation in cold-start scenarios (both user and item), susceptibility to data sparsity effects, and limited ability to incorporate rich item metadata or textual information that could enhance recommendation quality.

2.1.3 Hybrid Approaches and Early Integration Strategies

Recognizing the complementary strengths of CBF and CF, hybrid recommender systems emerged to combine multiple recommendation strategies. Early hybrid approaches employed simple fusion techniques, including weighted linear combinations, switching strategies, and cascade methods (Li et al., 2023). Linear combination methods aggregate predictions from multiple recommenders using learned or predefined weights. While straightforward to implement, these approaches often fail to capture complex interactions between different recommendation signals. Switching strategies dynamically select between different recommenders based on confidence measures or data availability, providing adaptive behavior but potentially underutilizing available information sources.

More sophisticated hybrid approaches employ feature-level fusion, where diverse information sources are integrated during the feature extraction or model training phases rather than at the prediction stage. This paradigm enables models to learn complex interactions between content-based and collaborative signals, leading to improved performance compared to late-stage fusion approaches (Najafabadi et al., 2019; Chen et al., 2020).

2.2 Deep Learning Revolution in Recommender Systems

2.2.1 Neural Collaborative Filtering and Deep Architectures

The integration of deep learning techniques fundamentally transformed recommender systems by enabling sophisticated nonlinear modeling of user-item interactions. Neural Collaborative Filtering (NCF) pioneered the application of multilayer perceptrons (MLPs) to recommendation tasks, demonstrating substantial improvements over traditional matrix factorization approaches (He et al., 2017). NCF replaces the simple inner product operation in matrix factorization with neural networks capable of learning arbitrary user-item interaction functions. This flexibility enables the capture of complex, nonlinear patterns in user behavior that linear methods cannot detect. Extensions of NCF have incorporated various neural architectures, including convolutional neural networks (CNNs) for spatial feature extraction and recurrent neural networks (RNNs) for sequential modeling.

AutoRec and its variants demonstrated the effectiveness of autoencoder architectures for collaborative filtering by learning compressed representations of user-item interactions. These approaches naturally handle the sparsity inherent in recommendation data by reconstructing incomplete interaction matrices through bottleneck representations (Li et al., 2020).

Variational Autoencoders (VAEs) further advanced autoencoder-based approaches by incorporating probabilistic modeling and regularization techniques. Mult-VAE and Mult-DAE have shown particular success in top-N recommendation tasks by learning robust multinomial distributions over items (Shenbin et al., 2020).

2.2.2 Sequential and Session-Based Modeling

Sequential recommendation has emerged as a critical area where deep learning demonstrates clear advantages over traditional approaches. Recurrent Neural Networks (RNNs), particularly Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM) networks, effectively capture temporal dynamics in user interaction sequences (Noorian et al., 2024). GRU4Rec established the foundation for neural session-based recommendation by modeling user sessions as sequences and predicting the next item based on previous interactions. Subsequent improvements incorporated attention mechanisms and hierarchical architectures to better capture both short-term session patterns and long-term user preferences. Caser treats user interaction sequences as "images" and applies convolutional filters to capture both local and global patterns in user behavior, demonstrating competitive performance with reduced computational complexity compared to recurrent approaches.

2.2.3 Transformer-Based Architectures and Attention Mechanisms

The introduction of transformer architectures revolutionized sequential recommendation by enabling parallel processing of sequences and capturing long-range dependencies through self-attention mechanisms. SASRec (Self-Attentive Sequential Recommendation) demonstrated that self-attention could effectively model user preference evolution without the sequential processing constraints of RNNs (Chen *et al.*, 2022).

BERT4Rec adapted the bidirectional encoder representations from transformers (BERT) for recommendation tasks, showing that bidirectional modeling of user sequences could capture richer patterns than unidirectional approaches. The success of BERT4Rec inspired numerous extensions incorporating masked language modeling objectives and multi-task learning frameworks for recommendation (Noorian *et al.*, 2024). Attention mechanisms have become ubiquitous in modern recommender systems, extending beyond sequential modeling to general user-item interaction modeling. Attention-based collaborative filtering methods dynamically weight the importance of different user interactions or item features, enabling more flexible and interpretable recommendation models.

However, existing transformer-based approaches exhibit a critical limitation: they typically apply attention mechanisms within single modalities or during late-stage fusion, missing opportunities for fine-grained cross-modal interaction during early feature learning phases. This limitation becomes particularly pronounced in sparse data scenarios where comprehensive multimodal integration is essential for maintaining recommendation quality. Most existing multimodal approaches suffer from fundamental architectural limitations: they process different data modalities through separate pipelines and combine them only at late stages, preventing the capture of complex cross-modal interactions essential for robust recommendation performance. This limitation becomes particularly acute in sparse data scenarios where comprehensive utilization of all available information sources is crucial.

2.3 Graph-Based Recommendation Systems

2.3.1 Graph Neural Networks for Recommendation

Graph Neural Networks (GNNs) have emerged as powerful tools for recommendation by explicitly modeling the relational structure inherent in user-item interactions. Unlike traditional approaches that treat interactions as independent observations, GNNs leverage graph structures to propagate information through networks, capturing high-order connectivity patterns crucial for understanding complex user-item relationships (Najafabadi *et al.*, 2025).

Graph Convolutional Networks (GCNs) pioneered the application of graph-based methods to collaborative filtering. GC-MC (Graph Convolutional Matrix Completion) formulates recommendation as a link prediction problem on user-item bipartite graphs, demonstrating the effectiveness of graph convolution operations for capturing collaborative signals (Cui *et al.*, 2024).

Neural Graph Collaborative Filtering (NGCF) advanced GCN-based approaches by explicitly modeling high-order connectivity through embedding propagation layers. NGCF captures the collaborative signal by recursively aggregating embeddings from higher-order neighbors, enabling the discovery of complex user-item relationships that traditional methods might miss (Wang *et al.*, 2022).

2.3.2 Heterogeneous Graph Approaches

Traditional GNN approaches for recommendation typically operate on homogeneous graphs with single node and edge types. However, real-world recommendation scenarios involve heterogeneous relationships between diverse entity types (users, items, categories, attributes), motivating the development of heterogeneous graph neural networks.

HetGNN and its variants enable modeling multiple entity and relationship types within unified graph structures by employing type-specific aggregation functions and semantic-aware attention mechanisms, effectively handling heterogeneous information. Additionally, metapath-based approaches utilize

predefined semantic paths to guide information propagation in heterogeneous graphs, capturing complex semantic relationships between users and items through intermediate entities like categories or brands (Najafabadi et al., 2025; Do et al., 2023). Despite their effectiveness, GNN-based approaches face significant scalability challenges in large-scale recommendation scenarios (Najafabadi et al., 2025). The computational complexity of neighborhood aggregation grows substantially with graph size, potentially limiting real-time applicability. Recent research has focused on sampling strategies, mini-batch training, and distributed graph processing to address these scalability concerns.

2.4 Large Language Models and Text-Based Recommendation

The emergence of Large Language Models (LLMs) has introduced new paradigms for recommendation systems, particularly in scenarios involving rich textual information. Unlike traditional ID-based approaches that represent users and items as abstract identifiers, LLM-based methods leverage textual descriptions and natural language interactions to generate recommendations.

P5 (Personalized Prompt & Predict Paradigm) pioneered the application of text-to-text generation models to recommendation tasks. By formulating various recommendation problems as text generation tasks with appropriate prompts, P5 demonstrated the potential for unified recommendation frameworks that can handle multiple tasks within a single model architecture. ChatGPT and instruction-following models have shown promising results in conversational recommendation scenarios, where users can express preferences and constraints in natural language. However, general-purpose LLMs often lack domain-specific knowledge and may struggle with the specialized requirements of recommendation tasks (Liu et al., 2023).

RecGPT represents a significant advancement as the first domain-adapted and fully-trained large language model specifically designed for text-based recommendation. Unlike general-purpose LLMs that require extensive fine-tuning or prompt engineering, RecGPT-7B is pre-trained on 20.5B tokens of recommendation-specific data, enabling superior performance on rating prediction and sequential recommendation tasks (Ngo & Nguyen, 2024). The success of RecGPT demonstrates several key insights for LLM-based recommendation: (1) domain-specific pre-training significantly outperforms general LLMs adapted through parameter-efficient methods, (2) text-based item representation aligns better with LLM capabilities compared to traditional ID-based approaches, and (3) instruction-following variants can effectively handle diverse recommendation tasks through appropriate prompt design.

Challenges in LLM-based recommendation include computational intensity, potential hallucination of non-existent items, and limited integration with collaborative filtering signals. These limitations highlight the need for hybrid approaches that combine the semantic understanding capabilities of LLMs with the collaborative intelligence of traditional recommendation methods.

2.6 Critical Research Gaps and Motivation for BEST-Rec

Based on our comprehensive analysis of existing literature, several critical research gaps persist in current recommender systems that directly motivate the development of BEST-Rec. Most existing approaches process heterogeneous data modalities—textual descriptions, numerical attributes, and collaborative signals—independently or through simple concatenation strategies, preventing the capture of complex cross-modal interactions essential for robust recommendations, particularly in sparse scenarios where leveraging all available information sources becomes crucial (Wang et al., 2022; Kharidia et al., 2024). While attention mechanisms have been widely adopted, most systems apply them within single modalities or at final fusion stages, missing opportunities for fine-grained cross-modal alignment during feature learning, as current cross-attention mechanisms typically operate on static representations rather than enabling dynamic alignment between user preferences and item characteristics throughout the learning process. Recent transformer-based approaches lack unified

architectures capable of jointly modeling textual semantics, numerical features, and collaborative signals within a cohesive attention framework, resulting in fragmented systems that limit the potential for synergistic learning across different information sources (Shuai et al., 2022). Current methods struggle to effectively leverage all available information sources in sparse scenarios, where traditional collaborative filtering fails due to insufficient interaction data, content-based methods cannot capture collaborative intelligence, and existing hybrid approaches often employ simplistic fusion strategies that fail to maximize the complementary benefits of different information sources (Hasan et al., 2024). Additionally, most recommendation systems assume static user preferences or employ simple temporal modeling, lacking dynamic preference alignment mechanisms that can adapt to evolving user interests in real-time, which represents a significant limitation for modern recommendation applications. Finally, while sophisticated models like heterogeneous GNNs and large language models achieve superior performance, their computational requirements often limit real-world deployment, particularly for real-time recommendation scenarios, creating a need for architectures that balance performance with practical scalability considerations. BEST-Rec, our algorithm explicitly addresses these limitations through dual complementary embeddings (combining BERT-based semantic and collaborative SVD embeddings), early-stage bidirectional cross-attention mechanisms, and a robust composite loss function that integrates regression and classification for improved accuracy and robustness.

2.7 Comprehensive Method Comparison

The following table provides a comprehensive comparison of major recommendation approaches, highlighting their strengths, limitations, and performance characteristics:

Table 1 summarizes the comparative analysis, highlighting BEST-Rec’s unique strengths

Method Category	Core Approach	Strengths	Limitations	Best use Cases
Traditional CF	Matrix Factorization	Efficiency, interpretability	Cold-start, limited integration	Stable domains with sufficient user-item interaction data
Deep Learning	Neural architectures	Nonlinear modelling, deep feature extraction	Large dataset requirement	Large-scale platforms with rich interaction data
Graph-Based	Graph convolution	Captures relational, structural interactions	Scalability challenges	Social networks, knowledge graphs
Heterogeneous GNN	Multi-type node/edge modeling	Rich relationship modeling, multi-modal integration	Engineering complexity, parameter sensitivity	Complex domains with diverse relationships
Transformer-Based	Self-attention	Sequential modeling, long-range dependencies	Limited cross-modal integration	Sequential recommendation, session-based scenarios
LLM-Based	Text embeddings	Cold-start robustness	Computational intensity	Text-rich domains, conversational recommendation
BEST-Rec	Dual embeddings, cross-attention	Comprehensive multimodal integration	Computational scalability considerations	Sparse scenarios, multi-source data integration

3. Methodology

BERT-Embedded Self-attention Transformer Recommender (BEST-Rec): Our proposed model, BEST-Rec, integrates pre-trained BERT text embeddings with a transformer based architecture to tackle data sparsity and cold-start issues in recommendations. The key idea is to enrich user and item representations with content features (both qualitative and quantitative data) and then fuse them via self-attention and cross-attention mechanisms. By jointly leveraging textual context (using BERT embedding) and collaborative signal, from historical interactions encoded via latent factors), BEST-Rec can make accurate predictions even when interactions are sparse or an item/user is new

3.1 Problem Formulation and Notation

Let $U = \{u_1, \dots, u_m\}$ be users and $I = \{i_1, \dots, i_n\}$ be items. Let $R \in \mathbb{R}^{m \times n}$ be a sparse interaction/rating matrix where

$R[u, i] = r_{ui}$ if interaction between u, i is observed, and 0 otherwise.

Denote the set of observed indices by $\Omega = (u, i): r_{ui} \text{ observed with } |\Omega| \ll mn$. Each user u has a review corpus $R_u = \{\text{free-text reviews written by } u\}$. Each item i has textual metadata t_i (e.g. title/description) and numeric attributes $n_i \in \mathbb{R}^3$ (price, average rating, rating count)

Objective. Learn a function

$$f_\theta: (u, i) \mapsto (\hat{r}_{ui}, \hat{c}_{ui})$$

That predicts a rating $\hat{r}_{ui} \in [1, 5]$ (regression head) and preference label

$\hat{c}_{ui} \in \{0, 1\}$ (classification head)

Cold-start & sparsity definition

- Sparsity (density): $p = \frac{|\Omega|}{mn}$; sparsity = $1 - p$
- Cold item: $|\{u: (u, i) \in \Omega\}| < 5$
- Cold user: $|\{i: (u, i) \in \Omega\}| < 3$
- Extreme cold user: $|R_u| = 0$ (no reviews)
- Extreme cold item: no interactions and no metadata text.

The table below summarised the notation used in this the paper

Table 2 summarizes the notation used for this and future chapters

Notation	Explanation
U, I	Set of all users U and all items I in the dataset
$u \in U, i \in I$	A specific user u and a specific item i .
r_{ui}	The rating value given by user u to item i (ground truth)
c_{ui}	A classification label derived from r_{ui} (e.g. 1 if u liked i , 0 otherwise)
t_i	Textual metadata of item i (e.g. the book title)
R_u	The set of review texts written by user u for items they rated.
e_i (item)	Embedding vector representation of item i (from t_i)
e_u (user)	Embedding vector representation of user u (aggregated from R_u)
f_θ	The feature fusion recommendation model with parameters θ
$\hat{r}_{ui}$	The rating score predicted by the model f_θ for user u on item i
$\hat{c}_{ui}$	The predicted classification output (e.g. probability user u likes item i).
L_{reg}, L_{cls}	Regression loss and classification loss components, respectively.
L_{total}	Composite total loss used for training the model

k	Number of latent factors retained after SVD
$n_i \in R^3$	Normalised numeric attributes of item i (average rating, rating-count, price)
$s_i \in R^k$	Latent vector of item i obtained from truncated SVD
x_i	Full item feature vector $[e_i n_i s_i]$
Ω	The set of observed user-item interaction (rating)

3.2 BEST-Rec Architecture Overview

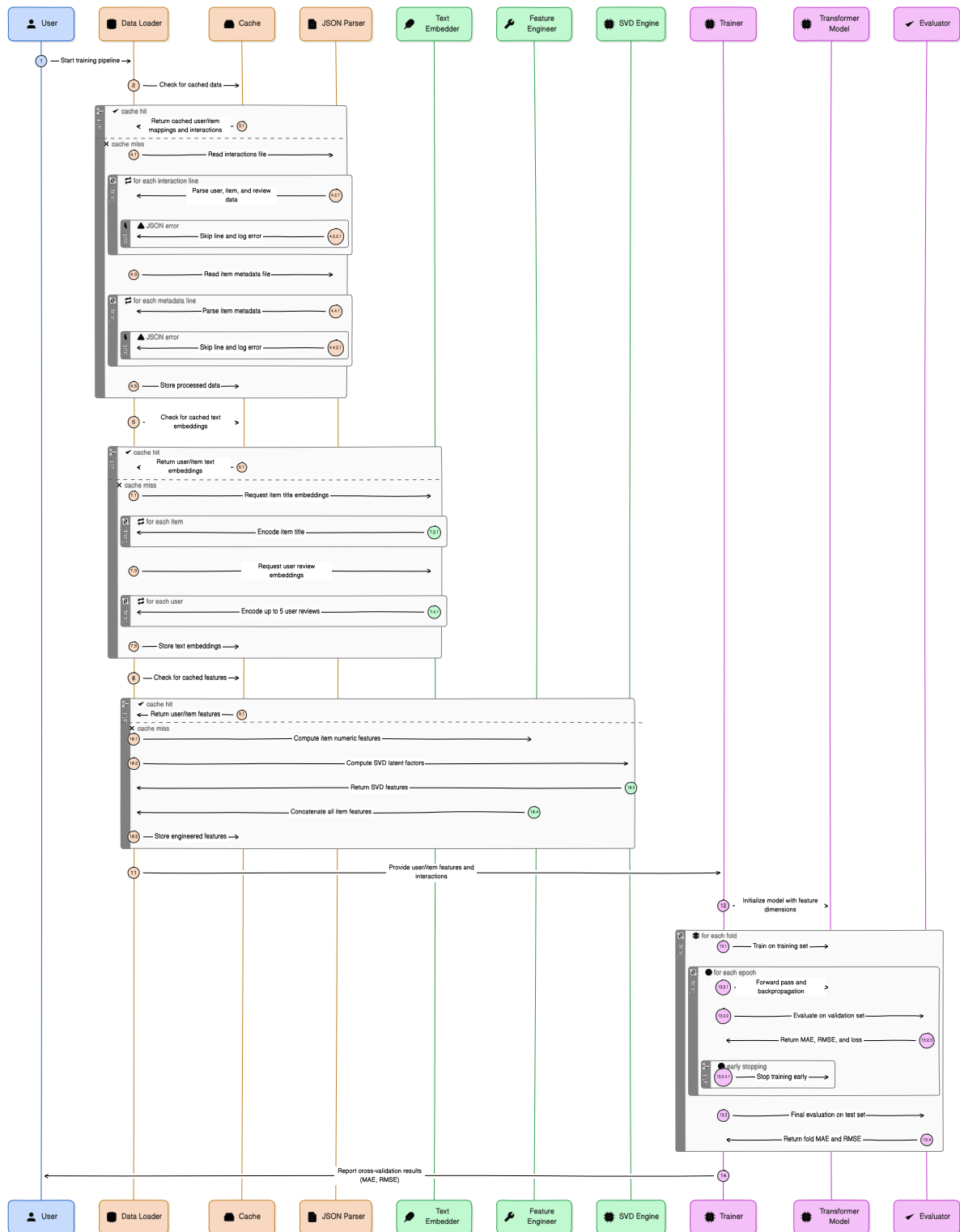


Figure 3.1. Overview of the model architecture for the transformer-based feature fusion recommender. The user's review text are encoded by a DistilBERT transformer to produce individual review embeddings, which are then aggregated into a single user embedding vector. DistilBERT likewise encodes the item's title text to produce the item embedding. These two embeddings are fed into a feature fusion model (neural network) that outputs a predicted rating (regression output) and a predicted preference (classification output). Arrows indicate the flow of information. The architecture uses pre-trained transformers for text encoding and a learned neural network for combining features.

BEST-Rec is a transformer-based feature-fusion recommender that jointly encodes user and item information and then aligns them with cross-attention before prediction. As shown in Figure 3.1, raw interactions and metadata are first preprocessed and cached; textual fields are embedded with DistilBERT; numeric attributes and collaborative signals (item SVD factors) are concatenated; the resulting vectors are linearly projected and lightly contextualized with mini transformer encoders; a user $\rightarrow$ item cross attention module performs fine-grained alignment; finally, a fusion MLP feed two prediction head.

Inputs.

For a user-item pair (u, i) , we build:

- A user text embedding $e_u \in R^{d_B}$ by averaging DistilBERT [CLS] vectors.
- An item representation $x_i = [e_i^{text}; n_i; s_i]$, where $e_i^{text} \in R^{d_B}$ is the DistilBERT title embedding, $n_i \in R^3$ are z-scored {avg_rating, rating_num, price}, and $s_i \in R^k$ are the item's truncated-SVD factors from the item times user rating matrix

Per-tower encoding.

User and item inputs are independently projected to a shared hidden space R^{d_H} and passed through shallow transformer encoders, yielding $h_u, h_i \in R^{d_H}$.

Cross-attention fusion.

To align preferences with content under sparsity, we let the user query the item representations:

$$Q = h_u W_Q, K = h_i W_K, V = h_i W_V, c_{(u \leftarrow i)} = \text{softmax}\left(\frac{QK^{(T)}}{\sqrt{d_k}}\right)V$$

The fused vector is

$$z = \phi(W_f[h_u; c_{u \leftarrow i}] + b_f),$$

where ϕ is ReLU activation function.

Dual prediction heads.

From z , the regression head outputs the predicted rating $\hat{r}_{ui} \in [1,5]$, and the classification head outputs a preference score $p_{ui} \in \{0,1\}$. The two heads share the lower layers, encouraging a representation that is both precise (regression) and robust (classification)

BEST-Rec operates through four interconnected components that address the fundamental limitations of existing approaches:

1. **Multi-modal embedding strategy.** We encoded item titles (and optional descriptions) and user reviews with DistilBERT; we z-score transform numeric attributes and we add low-rank SVD item factors from the item-by-user rating matrix. This unifies heterogeneous signals early, not just at the end
2. **Bidirectional cross attention mechanism.** We align user preference vectors with item representations during feature learning (not only after independent encoding). This combats sparsity by letting the model focus on the few item attributes most relevant to a user's textual profile.
3. **Feature-fusion network** A lightweight MLP fuses the attended pair-wise representations into a single vector suitable for both regression and classification heads.

4. **Dual head & composite loss.** Smooth- L_1 for rating and cross-entropy for labels yields robust training and regularizes the regression target.

Together, these choices fill a gap in prior work that either (i) processes modalities in isolated branches with late concatenation, or (ii) attends within a single modality only. Our early, cross-modal alignment is particularly effective under extreme sparsity/cold start

3.3 Multi-Modal Embedding Strategy

BEST-Rec transforms heterogeneous inputs – free-text reviews, item metadata, and collaborative interaction signals into a unified vector space that can be aligned and fused downstream (Fig. 3.1). Concretely, each user is represented by a transformer-based textual profile, and each item is represented by the concatenation of (i) a transformer text embedding, (ii) normalized numeric attributes, and (iii) a low-rank collaborative vector derived from truncated SVD. All text and SVD features are precomputed and cached to avoid re-computation during training

3.3.1 Textual Embedding via DistilBERT

Rationale. Most information attributes are qualitative (English text mostly). We therefore use a pretrained transformer encoder to convert text into dense vectors. DistilBERT was chosen to be the encoder since it offers an excellent accuracy-efficiency trade-off, retaining ~97% of BERT’s language understanding while being ~40% smaller and ~60% faster, which is crucial for a large volume of data.

Tokenization & encoding. Text is tokenized with wordPiece, mapped to embeddings, and contextualized by stacked self-attention blocks. We do not use a task-specific classification head; instead we take the contextual [CLS] representation as a sentence-level embedding.

- **Item text.** For item I with title (and optional description) t_i , DistilBERT yields $H_i \in R^{L \times d_B}$. We take

$$e_i^{text} = H_i[0] \in R^{d_B}, d_B = 768$$
- **User text.** For user u with review set $R_u = \{r_u^{(1)}, \dots, r_u^{(M)}\}$, we encode each review to a vector $z^{(j)} \in R^{d_B}$ (the review’s [CLS] embedding). We aggregate by mean pooling:

$$e_u^{text} = \frac{1}{M} \sum_{j=1}^M z^{(j)}$$

Mean pooling is simple, parameter-free, and robust in practice; it assumes each review contributes equally. (We also evaluated recency/length weighting; mean pooling provided stable performance without extra hyperparameters)

3.3.2 Structured Metadata Encoding

Each item carries numeric attributes that correlate with perceived quality and popularity. We use

$$n_i = [\text{avg_rating}, \text{rating_num}, \text{price}] \in R^3$$

Standardized feature-wise (z-score) across items. Missing values are imputed with 0 after standardization. This small numeric vector complements text with coarse but informative signals.

We selected the numeric features for the item by experimenting different combination of attributes, and found that those three attributes are most efficient in helping the model improve its performance.

3.3.3 Collaborative signal via Truncated SVD

To inject collaborative structure without a heavy collaborative filtering model, we factorize the sparse item-user matrix. Let

$$M \in R^{(n \times m)}, M[i, u] = \begin{cases} r_{ui}, & (u, i) \in \Omega \\ 0, & \text{otherwise} \end{cases}$$

With $n = |I|, m = |U|$ and observed indices Ω . We compute a rank-k truncated SVD:

$$M \approx U_k \Sigma_k V_k^{(\top)}, U_k^{(\top)} U_k = I, V_k^{(\top)} V_k = I$$

And define the item collaborative vector as

$$s_i = (U_k \Sigma_k)[i, :] \in R^k.$$

In our experiments we set $k=2000$ to preserve substantial variance while keeping inference lightweight. This low-rank factor captures global co-preference structure and denoises the raw interaction matrix, playing the role of a compact collaborative filtering signal that complements semantics from text.

3.4 Bidirectional Cross-Attention Mechanism

After multi-modal embedding process (refer to section 3.3), the user vector $e_u \in R^{d_B}$ and the item vector

$$x_i = [e_i^{text}; n_i; s_i] \in R^{d_B+3+k}$$

Are projected into a shared hidden space and lightly contextualized by per-tower transformer encoders

$$h_u = \text{Enc}_u(\text{LN}(W_u e_u + b_u)), \quad h_i = \text{Enc}_i(\text{LN}(W_i x_i + b_i))$$

With $W_u \in R^{d_H \times d_B}, W_i \in R^{d_H \times (d_B+3+k)}$, layer norm LN, 2-layer Transformer encoders, and hidden size $d_H = 512$. In the current implementation each tower receives a single token; the Transformer encoders therefore act as learned feed-forward residual blocks with self-attention weights collapsing to identity, but crucially they **learn tower-specific projections and normalization** that benefit downstream attention.

We then perform **cross-attention** with the user as **query** and the item as **key/value** (user->item). For h heads (we use $h=4$), head $a \in \{1, \dots, h\}$ computes

$$Q_a = h_u W_Q^{(a)}, \quad K_a = h_i W_K^{(a)}, \quad V_a = h_i W_V^{(a)}$$

with $W_Q^{(a)}, W_K^{(a)}, W_V^{(a)} \in R^{d_H \times d_k}$ and $d_k = \frac{d_H}{h}$. Scaled dot-product attention per head is

$$A_a = \text{softmax}\left(\frac{Q_a K_a^\top}{\sqrt{d_k}}\right), \quad c^{(a)} = A_a V_a$$

Heads are concatenated and projected:

$$c_{u \leftarrow i} = ||_{a=1}^h c^{(a)} W_O, \quad W_O \in R^{(h d_k) \times d_H}.$$

Because each tower is **bidirectional**, with DistilBERT upstream for text and a bidirectional Transformer encoder per tower), the cross-attention aligns contextualized user preference h_u with contextualized item representations h_i , yielding a fine-grained, pair-specific summary $c_{u \leftarrow i}$. This is particularly helpful in sparse regimes: attention re-weights the few reliable signals (e.g., salient title terms, SVD directions correlated with the user's profile) instead of treating all item features equally.

Rather than flattening features by concatenation, attention re-weights which parts of an item representation matter for a given user (and vice versa), amplifying the few informative signals available in sparse regimes.

3.5 Feature Fusion and Prediction Heads

Let h_u denote the user output and $c_{u \leftarrow i}$ the cross-attended item summary from section 3.4. We build a fused representation via a gated linear projection and nonlinearity:

$$z = \phi(W_f[h_u; c_{u \leftarrow i}] + b_f) \in R^{d_H},$$

Where $[\cdot; \cdot]$ is concatenation, $W_f \in R^{d_H \times 2d_H}$, $b_f \in R^{d_H}$, and $\phi = ReLU$.

We employ dual prediction heads that share the fused backbone z :

Rating regression head (continuous 1-5).

$$\widehat{r}_{ui} = w_r^T z + b_r, \quad w_r \in R^{d_H}, \quad b_r \in R$$

Preference classification head.

Multi-class rating: logits $o \in R^5$ with softmax; training labels $y = \text{int}(r) - 1$

The classification head regularizes the shared representation: optimizing a coarse “like/dislike” (or class) decision guides the regressor toward better calibrated scores, often reducing variance and outlier sensitivity in sparse regimes (multi-task learning effect)

Calibration/inference. For application that need probabilities, use $\text{softmax}(o)$ to obtain p_{ui} . At inference, ranking can use $\widehat{r}_{ui}, p_{ui}$ or a calibrated combination

3.6 Training Objective and Optimization

3.6.1 Composite Loss

We jointly optimize a regression loss and a classification loss:

$$L = \frac{1}{|B|} \sum_{(u,i) \in B} \text{SmoothL1}(\widehat{r}_{ui}, r_{ui}; \delta) + \lambda \frac{1}{|B|} \sum_{(u,i) \in B} CE(o_{ui}, y_{ui})$$

We use $\lambda = 1.5$ in our implementation. The *Smooth* – L_1 (Huber) loss with threshold δ behaves quadratically for small errors and linearly for large ones, improving robustness to outliers in noisy rating. CE is standard cross-entropy for either 5-class ratings.

3.6.2 Training strategies

With the data processed and feature extracted, we train the recommendation model f_θ using a batch training strategy. We implemented the model training using PyTorch, which provides convenient tools like the DataLoader for batching and shuffling. Mini-batch training allows the model to see multiple examples at once, making training a large dataset more tractable and stabilize the gradient update.

Batch Training with DataLoader: We form training batches of a fixed size. Each batch is a set of user-item pairs sampled from the training set. The DataLoader shuffles the training data at the start of each epoch to ensure the model does not learn any spurious order, for each batch, the training loop performs the following operations:

1. **Batch Preparation:** For each interaction in the batch, retrieve the precomputed e_u and e_i from the cache. Assemble the batch of user embeddings into a matrix of shape (batch_size, d) and item embeddings into another (batch_size, d). Also prepare the target rating values and classification labels as vectors of length batch_size.
2. **Forward Pass:** Feed the batch embedding through the fusion network to obtain predictions. The model produces a vector of predicted ratings $\hat{r}_{ui}$ for the batch and a vector of predicted preference scores $\hat{c}_{ui}$. This forward computation is done by multiplying the inputs with the network weights using a matrix and applying non-linear activations as defined by the network architecture.
3. **Loss Computation:** Compute the loss for the batch by comparing predictions with ground truth. We use a composite loss function L_{total} that combines a regression loss and a classification loss as described in section 3.6.1 composite loss function
4. **Backward Pass and Optimisation:** We compute the gradient of L_{total} with respect to all model parameters θ via backpropagation. The optimiser then updates the model's weight in the direction that reduces the loss. Specifically, for each parameter θ_j , $\theta_j \leftarrow \theta_j - \eta \frac{\partial L_{total}}{\partial \theta_j}$, where η is the learning rate. We apply gradient clipping if needed to prevent exploding gradients. Adam was chosen for its ability to adapt the learning rate for each parameter and generally converge faster on deep networks. We note that since the DistilBERT encoders are fixed, the only parameters being updated in this loop are those of the fusion network.

The above process repeats for each batch and for multiple epochs. An epoch consists of going through all training batches. We monitor performance on the validation set at the end of each epoch. The validation performance is used for early stopping: if the model's validation loss has not improved for a certain number of epochs (patience), we stop training to prevent overfitting. Early stopping is a regularisation technique that halts training when further training would only make the model fit the noise in training data rather than improve generalization.

3.7 Computational complexity analyse

3.7.1 Model Training Pseudo code

The key architecture of the BEST-Rec model is described using the pseudo code below:

Transformer Fusion Recommender Function

inputs: user_f (B,d_B), item_f (B, d_B+3+k)

```

u = Linear(user_f)      # (B,d_H)
i = Linear(item_f)      # (B,d_H)

u = u.unsqueeze(1)      # (B,1,d_H)
i = i.unsqueeze(1)      # (B,1,d_H)

```



```

u = Encoder_L(u) × L      # (B,1,d_H) (# tokens = 1)
i = Encoder_L(i) × L

(attn, _) = MultiHeadAttn(query=u, key=i, value=i) # (B,1,d_H)

u = u.squeeze(1)          # (B,d_H)
c = attn.squeeze(1)

fused = Linear(concat(u, c)) # (B,d_H)

rating = MLP(fused)        # (B,1) → squeeze → (B,)
logits = Linear(fused)     # (B,5)

```

K-Fold Training Function

INPUT : interactions, user_feat, item_feat, F, B, E, device

OUTPUT: lists MAE[F], RMSE[F]

```

# build array of row indices 0..N-1
groups ← [interactions[i].user_id for i in 0..N-1]
splitter ← GroupKFold(n_splits=F)

INIT MAE_list ← [], RMSE_list ← []

FOR (fold, (train_idx, test_idx)) in enumerate(splitter.split(indices, groups)):
    train_rows ← [interactions[i] for i in train_idx]
    test_rows ← [interactions[i] for i in test_idx]

    train_ds ← InteractionDataset(train_rows, user_feat, item_feat)
    test_ds ← InteractionDataset(test_rows, user_feat, item_feat)
    train_dl ← DataLoader(train_ds, batch=B, shuffle=True)
    test_dl ← DataLoader(test_ds, batch=B, shuffle=False)

    model ← TransformerFusionRecommender(d_B,
                                          d_B+3+k,
                                          d_H,
                                          L,
                                          num_heads=4).to(device)
    optim ← Adam(model.parameters(), lr = 1e-4)
    best_mae ← +∞ ; patience ← 0

    FOR epoch in 1..E:
        train_loss ← train_epoch(model, optim, train_dl)
        (val_mae, val_rmse, _) ← evaluate(model, test_dl)

        IF val_mae < best_mae:
            best_mae ← val_mae
            patience ← 0
        ELSE:
            patience += 1
            IF patience ≥ 5: BREAK

    (fold_mae, fold_rmse, _) ← evaluate(model, test_dl)
    MAE_list.append(fold_mae)
    RMSE_list.append(fold_rmse)
RETURN {MAE_list, RMSE_list}

```

Training epoch

```

FOR each batch (user_f, item_f, r, cls) in train_dl:
    user_f, item_f, r, cls ← to(device)
    optim.zero_grad()

    (f, logits) ← model(user_f, item_f)

    loss_reg ← SmoothL1Loss(f, r)
    loss_cls ← CrossEntropy(logits, cls)
    total ← loss_reg + λ · loss_cls

    total.backward()
    optim.step()

```

3.7.2 complexity Analysis

In this section, the model architecture will be analysed with its time and space complexity to understand and predict the efficiency of how BEST-Rec will perform.

Preprocess: In data pre-process step each meta data is loaded once with time complexity of $\theta(I)$ where I is the number of item, and each interaction is loaded, with time complexity of $\theta(N)$, where N is the number of interactions. The space needed to load the data and performing mapping to dictionary is $\theta(U + I)$

SVD: The time complexity of SVD depends on both the number of interactions, users and the items metadata. The time complexity of SVD is $O(N \cdot K + (I + U) \cdot K^2)$ Where U is the number of users and K is the latent dimension of the SVD output, which is 2000 in our experiment

Model parameters: the model parameters that fits in our GPU take space about $(d_B + K) \cdot d_H + (4L + 5) \cdot d_h^2$ where d_H is the hidden width of the fusion network (512 in our experiment), L is the encoded layers per tower (2) and d_B is the DistilBERT hidden size (768)

Training Epoch: The training epoch takes time complexity of $O(0.8N \cdot 6 \cdot d_H^2)$, it takes 80% of the number of interaction for training because of the 5-fold cross validation, the cost per sample have multiple factor of 6 since we have two encoder feed forward network twice, with additional cross-attention $O(K^T)$ multiply and VO projection operation

Total Training Time: the total training time of the BEST-Rec model architecture will be $T_{total} = F \cdot E_{eff} \cdot T_{epoch}$ where F is the number of folds, E_{eff} is the average epoch run before early stop and T_{epoch} is the training epoch time discussed earlier.

By the time complexity analyse, we can ensures that when the number of interactions, users or items increased for bigger dataset or even with real life usage situation, the time and space used only increased linearly proportional to the number of increase, which ensure the scalability of our proposed model.

4. Experimental Setup

4.1 Dataset and Data Characteristics

The dataset used is a subset of the Amazon 2023 review dataset¹, a dataset collected on the Amazon shopping website containing verified user purchases, ratings, and reviews of items. The dataset has been divided into different e-commerce categories. We are going to

explore the categories of Beauty and Books to evaluate the proposed model in terms of accuracy, efficiency and computational cost.

Table 3. Summarise dataset information

Dataset	#Users	#Items	#Interactions	Sparsity	Cold user	Cold item
Beauty	631,986	112,565	701,528	99.9990%	622,827 (98.55%)	85,032 (75.54%)
Books	10,297,355	4,446,065	29,475,453	99.9999%	7,770,787 (75.46%)	3,429,970 (77.15%)

The majority of the users and items are in a cold state, meaning an item has not interacted with users very much, and a user has not interacted with many items. We define a cold item as items that have less than five interactions with users, and a cold user as a user that has not interacted with more than three item. Table 3 provides a summarisation about the dataset. We can see that most items have not interacted with more than 4 users, making the dataset suitable for simulating a cold-start problem to evaluate our models' capability in dealing with cold-start items.

Furthermore, the two dataset come from two very different domains in terms of scales and possibly different target user, this can help us test the adaptability of the our proposed model.

The raw data provided is in 2 separate JavaScript object notation (JSON) files as key-value pairs of information. One file provides the interaction between a user and an item, and the other provides the metadata we need, as auxiliary information, to incorporate and utilise in the recommendation system.

The attributes and shape of the user-item interaction raw data are summarised in the table below:

Table 4. Data structure for user-item interaction

Attribute	Type	Explanation	Example
Rating	Integer	Rating the product, discrete data which have value of either 1, 2, 3, 4 or 5	5
Title	String	Title of the user review, could be empty, meaning user leave a rating but not qualitative review	Such a lovely scent but not overpowering.
Text	String	The text body of the user review could be empty, meaning the user leaves a rating but not a qualitative review	This spray is really nice. It smells really good, goes on really fine, and does the trick. I will say it feels like you need a lot of it though to get the texture I want. I have a lot of hair, medium thickness. I am comparing to other brands with yucky chemicals so I'm gonna

			stick with this. Try it!
Images	List	Images that users post after they receive the product	[]
Asin	String	Specific ID of a particular Product	B00YQ6X8EO
Parent Asin	String	parentID of the product, Note: Products with different colours, styles, sizes usually belong to the same parent ID. The “asin” in previous Amazon datasets is actually parent ID. Please use parent ID to find product meta.	B00YQ6X8EO
User id	String	ID of the reviewer	AGKHLEW2SOWHNMFIJ GBECAF7INQ
Timestamp	Integer	Time of the review (unix time)	1588687728923
Verified purchase	Boolean	User purchase verification	True
Helpful vote	Integer	Helpful votes of the review	0

TABLE 5. Data structure of item metadata

Attribute	Type	Explain	Example
Main category	String	Main category of the product	All Beauty
Title	String	Name of the product	Howard LC0008 Leather Conditioner, 8-Ounce (4- Pack)
Average rating	Float	Rating of the product shown on the product page	4.8
Rating number	Integer	Number of rating in the product	10
Features	List	Bullet-point format features of the products	[]
Description	List	Description of the products	[]
Price	Float	Price in US dollar (at the time of data collection)	null
Images	List	Image of the product	[{"thumb": "https://m.media-amazon.com/images/I/41qfSfqNyL._SS40_.jpg", "large": "https://m.media-amazon.com/images/I/41qfSfqNyL.jpg", "variant": "MAIN", "hi_res": "null"}, {"thumb": "https://m.media-amazon.com/images/I/41w2yznfuZL._SS40_.jpg", "large": "https://m.media-amazon.com/images/I/41w2yznfuZL.jpg", "variant": "PT01", "hi_res": "https://m.media-amazon.com/images/I/71i77Aul9xL._SL1500_.jpg"}]

Videos	List	Videos of the product including title and url	[]
Store	String	Store name of the product	Howard Products
Categories	List	Hierarchical categories of the product	[]
Details	Object	Product details, including materials, brand, size, etc	{"Package Dimensions": "7.1 x 5.5 x 3 inches; 2.38 Pounds", "UPC": "617390882781"}
Parent asin	String	Parent ID of the product	B01CUPMQZE
Bought together	List	Recommended bundle from the website	null

4.2 Data Preprocessing Pipeline

The data shown in table 4 and table 5 in chapter 4.1 will be used as a JSON input, then data is split into 5 different folds to performs a 5-fold cross validation later on. This ensures the model evaluation is fair and perform on unseen data. The 5-fold is split using the KFold() function, with flag of shuffle be true and a random seed state of 42 to ensure the data is randomly sampled but the result is reproducible.

After the raw data is split up to 5 folds, it will be pre-process through the steps below

1. **Parsing & filtering:** drop malformed rows; ensure required IDs and rating are present; keep tokenizer parsable English text.
2. **ID mapping:** map raw user_id and parent_asin to contiguous indices for storage and fast lookup.
3. **Text embedding:** item's text field and user's text field is embedded with the DistilBERT (distilbert-base-uncased) model, to a 768-dimension vector
4. **Numeric features:** for each item i , form $n_i = [\text{avg_rating}, \text{rating_num}, \text{price}]$ and z-score each feature across items; missing values imputed with 0 after standardization
5. **Collaborative factors:** construct sparse item x user rating matrix M and compute truncated SVD (rank $k = 2000$); take $s_i = (U_k \Sigma_k)[i, :] \in R^{2000}$ as the item collaborative filtering vector
6. **Concatenation & caching:** assemble item feature $x_i = [e_i^{\text{text}}, n_i; s_i] \in R^{768+3+2000}$, cache $\{e_u^{\text{text}}, x_i\}$ to avoid recomputation

4.3 Baseline methods and implementation details

We chose the classic and recent recommendation algorithms to be the baseline of the evaluation. The models that we are comparing cover a wide range of different algorithms, including collaborative filtering-based methods, review information recommendation methods with deep learning, and review information methods based on attention mechanisms.

Matrix Factorisation [13], Matrix factorisation techniques are a key method in collaborative filtering recommendation systems. The model works by representing users and items as vectors in a latent factor space, where the interactions between users and items are

modelled as inner products of these vectors. The factorisation allows the system to predict unknown ratings based on the patterns of previously observed ratings. Each user's and item's latent factors are learned by minimising a regularised squared error function, ensuring the model avoids over fitting.

SVD [22], Singular Value Decomposition (SVD) used a matrix factorisation strategy in recommendation system to predict user preferences. The SVD algorithm decomposes the user-item interaction matrix into three matrices representing users, items and latent factors. By capturing the underlying relationships between users and items, SVD can effectively predict missing ratings and allow personalised recommendations.

ARPM [14], The Aspect-based Rating Prediction Method (ARPM) utilises user preferences extracted from review data to predict business ratings. It incorporates aspect detection and sentiment analysis to analyse the various aspects mentioned in reviews. Users' preferences are modelled using a feature vector that reflects their sentiment towards different aspects. The method uses matrix factorisation to predict ratings by finding similar users based on their preference and predicted rating for items.

FSER [30], The FSER (Fine-grained Sentiment for Explainable Recommendation) model combines fine-grained sentiment analysis with matrix factorisation to provide both accurate recommendations and intuitive explanations. By leveraging review data, the model constructs three key matrices: the user rating matrix, the user-aspect sentiment matrix, and the item aspect-descriptive word frequency matrix. The matrices are factorised jointly to predict user preference. At the same time, an explanation for the recommendations is generated, allowing the model to be interpretable and offering a clear reason behind the suggested item.

RACSMD [17], The RACSMD (Recommendation Algorithm Combining Sentiment and Matrix Decomposition) is a hybrid recommendation model that integrates sentiment analysis of user comments with traditional matrix decomposition techniques to improve recommendation quality. The algorithm's core is a Long Short-Term Memory (LSTM) network for sentiment analysis, capturing the emotional tone of user reviews. The values are then combined with user rating data to form a "user rating standard value", which accurately reflects a user's subjective evaluation.

ChatGPT (few shot) [19], the paper "Is ChatGPT a Good Recommender?" explores the potential of ChatGPT as a general-purpose recommender system. Unlike traditional recommendation systems that rely on specific training data and model fine-tuning for various tasks, the study uses prompts to convert the recommendations task, not a natural language task, evaluating ChatGPT on five scenarios: rating prediction, sequential recommendation, direct recommendation, explanation generation, and review summarisation.

WDL [2], The Wide & Deep Learning for recommender systems introduces a hybrid model combining wide linear models and deep neural networks for recommender system. This approach leverages the strengths of both memorisation and generalisation. Wide models memorise frequent feature occurrences, using cross-product transformation, while deep models generalise by learning a low-dimensional embedding for sparse features.

RGCL [28], the RGCL (Review-aware Graph Contrastive Learning) framework presents a novel method for improving review-based recommendation system. The approach utilises a graph structure formed by users, items, and their interactions, enriched by edge features extracted from reviews. RGCL introduces a review-aware graph learning module to effectively integrate review data into user-item embedding learning and interaction modelling.

P5 [7], The paper "Recommendation as Language Processing" introduces a unified recommendation framework called P5, which incorporates pertaining, personalised prompt and predictive paradigms to handle a wide range of recommendation tasks. This framework uses a text-to-text format, transforming user-item interactions, item metadata, reviews and user descriptions into natural language sequences.

Exp3RT [12], the Exp3rt model is a large language model-based recommender system designed to improve the accuracy of rating prediction and explainability for recommendations at the same time. It fine-tunes a student LLM by distilling reasoning capabilities from a teacher LLM. By performing the three key steps of preference extraction, profile construction and reasoning-enhanced rating prediction, the model effectively combines subjective preferences from reviews with objective item descriptions to generate more accurate predictions.

The baseline model we chose to compare can be categorised into for main recommendation methods:

- **Classic CF:** MF/SVD
- **Review-aware methods:** ARPM, FSER, RACSMD
- **General deep recommenders:** WDL, RGCL
- **Language-model-based recommenders:** P5, Exp3RT, ChatGPT-style prompting

Provenance & fairness: Results for **Exp3RT**, **ChatGPT**, and **RACSMD** are directly cited from the respective papers. Where possible, we match their metric definition and reported split protocols. Any baseline we implement locally follow the preprocessing, metrics, and cross-validation setup as BEST-Rec.

4.4 Evaluation Protocol and Metrics

The deep learning task for the model to perform is a regression task that predicts the review rating between 1 and 5. Since the results are discrete numeracy data, we use mean absolute error (MAE) and root mean squared error (RMSE) to measure how accurate the prediction is, relative to the actual rating.

The MAE measures the average of the magnitude of the error between a pair of predicted values and true labelled values without considering their direction. It indicates how large the

errors are on average, and it is easy to interpret, as the unit will be the same as the true label's unit. In our case, the error will be the absolute difference in the rating.

The mathematical formula of an MAE is given below:

$$\text{MAE} = \frac{1}{|\Omega_{test}|} \sum_{(u,i) \in \Omega_{test}} |r_{ui} - \hat{r}_{ui}|$$

The goal is to minimise the error between our model's prediction and the actual value. Hence, a smaller MAE means we have a more accurate model.

The RMSE measures the average of the squares of the error, giving more weight and penalties to larger errors. It squares the difference between the actual values before averaging them, making the RMSE metric more sensitive to outliers and emphasising cases where predictions are far from the actual value.

The mathematical formula of RMSE is given below:

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{test}|} \sum_{(u,i) \in \Omega_{test}} (r_{ui} - \hat{r}_{ui})^2}$$

The use of both MAE and RMSE allows us to understand what kinds of error and distribution the prediction has, since MAE treats all errors equally while RMSE gives more weight to larger errors. By analysing the difference between the two values, we could better understand the prediction error our model has. Furthermore, in the model tuning stage, the error could guide us about model improvement.

A low MAE can indicate consistent error reductions, whereas a low RMSE can show improvements in handling large errors effectively. Considering both metrics when evaluating the model could help us get a more comprehensive evaluation of the model's accuracy in rating prediction.

Additionally, to ensure the validity of our experiment, a 5-fold cross-validation (CV) will be used for experimenting. A 5-fold CV will randomly split the user-item interaction data into 5 different chunks, each of which will have the same amount of data. The training process will take four folds to learn the pattern. Using the left out 1 fold to evaluate the model is equivalent to training on 80% of the dataset and evaluating the 20% as unseen data, simulating the real-world prediction. A 5 fold CV is a commonly used method for evaluating a model, as it provides a well-balanced amount of training and evaluating data. At the same time, we can evaluate the consistency of the result, and how training and evaluating on the same dataset can differ.

4.5 Implementation Details and Reproducibility

The experiments are conducted on a NVIDIA L4 GPU, with 16 vCPU and 62 GB Random Access Memory (RAM). The model architecture is kept consistent with the rating prediction algorithm as described in Section 3.2 BEST-Rec model architecture. The experiments are conducted with the software version listed below.

- Python 3.10.12
- Torch 2.1.0+cu118

- Torch-cluster 1.6.3+pt21cu118
- Torch-geometric 2.6.1
- Torch-scatter 2.1.2+pt21cu118
- Torch-sparse 0.6.18+pt21cu118
- Torch-spline-conv 1.2.2+pt21cu118
- Pyg-lib 0.0+pt21cu118
- Transformers 4.51.3
- Tokenizers 0.21.1
- Sentence-transformers 4.1.0
- Tqdm 4.67.1
- Scikit-learn 1.6.1
- Scipy 1.15.2
- Pandas 2.2.3
- Numpy 2.2.5

Reproducibility. the experiment is run with fixed seeds, and explicit hyperparameters states in section 4.6 to ensure the reproducibility of the experiment.

4.6 Hyperparameter Configuration and Tuning

The hyper parameters and model configuration in the experiments are shown in table 6 below.

TABLE 6 Parameter used in the experiment

Parameter	Value
Learning rate (η)	1×10^{-4}
SVD embedding size	2000
Optimisers	ADAM
Batch size	2048
Loss function	Smooth L1 and Cross Entropy
Training epoch	15
Hidden dimension	512
Transformer encoders	2 layers/tower, 4 heads

We validate, with a grid search of $k \in \{256, 512, 1024, 2000\}$ and $d_H \in \{256, 512\}$ on amazon beauty dataset; with $k=2000$ and $d_H = 512$ provided the best performance.

SECTION 5: RESULTS AND ANALYSIS

5.1 Overall Performance Comparison

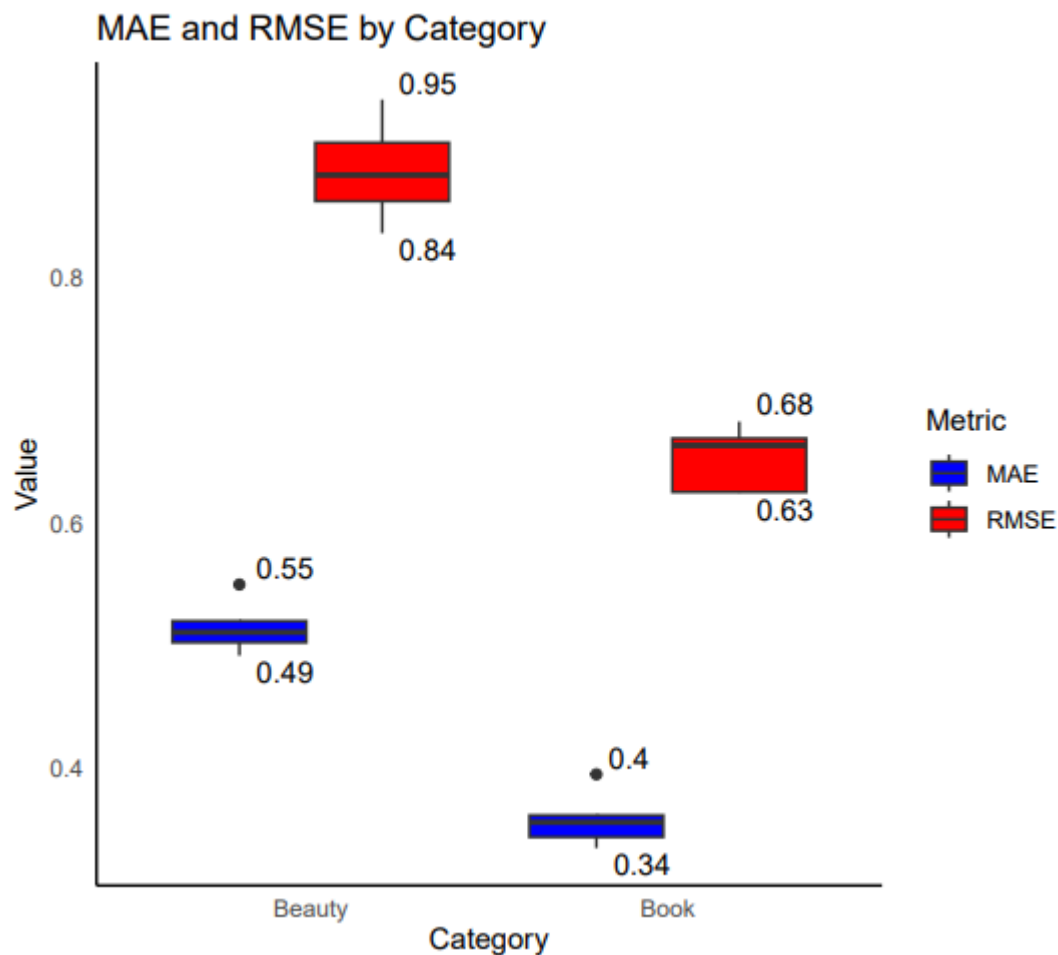


Figure 5.1, MAE and RMSE result from 5-fold cv. The blue box represents the value of the MAE metric, and the red box represents the RMSE metric.

The results of our modes are as follow:

Table 6 MAE and RMSE result from each of the 5-fold CV

	MAE			RMSE		
	5-fold CV results	Mean	95 % CI	5-fold CV results	Mean	95 % CI
Beauty	[0.4921, 0.5204, 0.5502, 0.5029, 0.5112]	0.5153	0.515 ± 0.027	[0.8844, 0.9109, 0.9460, 0.8631, 0.8370]	0.8883	0.888 ± 0.053
Books	[0.3441, 0.3561, 0.3953, 0.3618, 0.3350]	0.3585	0.358 ± 0.029	[0.6257, 0.6695, 0.6832, 0.6637, 0.6251]	0.6534	0.653 ± 0.033

In table 6, The individual result in each fold, in order, is shown on the 5-fold CV column. The mean and 95% confidence interval (CI) of result is shown to validate the significance and the consistent of the data by showing the low variance.

From Figure 5.1, we can derive a few key insight about our model:

Firstly, from the result of the 5-fold CV, the performance within the same dataset has a small variance. This means our model is consistent in terms of performance each time it makes a prediction, and hence, proves the validity and reliability of the model's rating prediction.

Secondly, the difference between the datasets for both MAE and RMSE metrics is minimal. For example, in the MAE metric, the average value difference between the Books and All_Beauty dataset is only 0.16, which shows that the model works for different categories consistently and proves the flexibility of our model.

Lastly, the value of RMSE is consistently higher than MAE. This inconsistent result between RMSE and MAE shows that our model generally performs really well, with the exception of large differences as outliers, as the root mean squared error will square the difference and penalise more for larger prediction errors.

The following section compares our model with the baseline model described in section 4.3

Table 7 Summary of the results comparing:

Beauty Dataset:			Books Dataset:		
Method	RMSE	MAE	Method	RMSE	MAE
MF	1.1973	0.9461	MF	0.6683	0.4572
SVD	0.9593	0.7784	WDL	0.7020	0.5324
APRM	1.1221	-	RGCL	0.7562	0.6227
FSER	1.1604	-	P5	0.8868	0.4756
RACSMD	1.1024	-	Exp3RT	0.6508	0.4297
ChatGPT	1.0751	0.6977	BEST-Rec	0.6534	0.3585
BEST-Rec	0.8883	0.5153			

Firstly, the models perform differently on those two Amazon datasets, but we can still define the trends of model accuracy. KNN has the worst performance. This could possibly be because the model does not consider any auxiliary factors and the embedding space for KNN is not well presented for the raw data; hence, in the task of rating prediction, it could lead to inaccurate prediction; furthermore, it could also be suffering from the sparse dataset we have.

In addition, the sentiment-analysis-based models WDL, RGCL, P5, Exp3RT, and our model have similar performance on the Books dataset. However, compared to the baseline models in the All_Beauty dataset, our model has a significant performance improvement compared to other baseline models. This shows that using a sentiment analysis to extract extra information can effectively improve the performance compared to only using the quantitative data.

Finally, our model achieved the best performance in various datasets with both MAE and RMSE metrics All_Beauty dataset, and performed the best in MAE and second best in RMSE with a minor difference in the Books dataset. The result is clearly shown in Table 7; it proves that our model is efficient and effective in solving the rating prediction problem. There are several reasons why our model achieved the best performance: the BERT embedding successfully captures the meaning in the descriptions of items, the SVD embedding on users and items successfully reduces noise and extracts features well, and the composite loss function forces the model to better understand the relationship between user and items with a general guide of classification method to prevents overfitting problems for the model.

5.2 Ablation Studies / Component Analysis

In this ablation study, we aim to find out how each component in the model helps to achieve the high performance using the beauty dataset. We will evaluate the effectiveness of the model with 2 different aspects: the data and the components in the model architecture.

TABLE 8 results for alternative model in ablation study

	MAE		RMSE	
	5-fold CV results	Mean	5-fold CV results	Mean
No_SVD	[0.5242, 0.5203, 0.5128, 0.5006, 0.5184]	0.5153	[0.8894, 0.8415, 0.9113, 0.8530, 0.9253]	0.8840
No_Text	[1.0188, 0.9898, 0.9945, 0.9699, 0.9785]	0.9903	[1.3918, 1.412, 1.4042, 1.4535, 1.424]	1.4171
No_Numeric	[0.5038, 0.534, 0.5726, 0.4926, 0.5106]	0.5226	[0.885, 0.8763, 0.9596, 0.8971, 0.88]	0.8996
No_Cross	[0.529, 0.529, 0.521, 0.535, 0.515]	0.5260	[0.884, 0.880, 0.916, 0.955, 0.886]	0.9040
GPT-embed	[0.5971, 0.5214, 0.5656, 0.5933, 0.5206]	0.5596	[1.0127, 0.9007, 0.8894, 1.0121, 0.9088]	0.9447

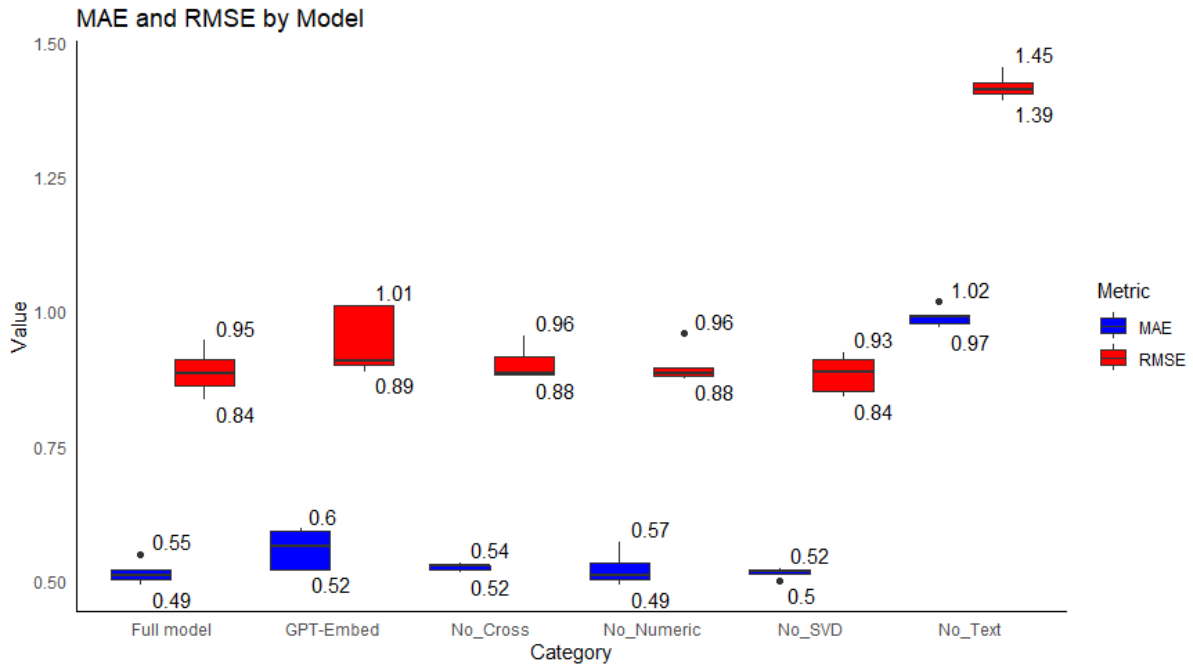


Figure 5.2, MAE and RMSE result from 5-fold CV for each variant model experiment on the beauty dataset. The blue box represents the value of the MAE metric, and the red box represents the RMSE metric.

In the evaluation of how auxiliary data information could affect the accuracy of our model's prediction. We make 2 variants of datasets, no-text and no-numeric, to evaluate how the data could affect our result.

In the no-text dataset, we feed only the quantitative data into the full model for rating prediction instead of the whole dataset. The result of using the no-text dataset achieved an average MAE of 0.9903 and an average RMSE of 1.4171 over 5-fold cross-validation, which is a significant increase of the MAE and RMSE value, meaning that the performance has significantly decreased; From the result, we can conclude that the qualitative data are useful for rating prediction, but furthermore, our model is able to extract the semantics of the review information correctly and can use the information to enhance the performance of rating prediction task.

In the no-numeric dataset, we only feed the qualitative data for our model to learn and predict the rating. The result of using the no-numeric dataset achieved an average MAE of 0.5226 and an average of RMSE of 0.8996, which is slightly worse performance than the full dataset, indicates that the quantitative data does not affect as much in our model's evaluation, but it still assists and guide our model towards the right direction.

On evaluating how each component affects the final results, we designed variants of the original model, no_cross, no_SVD and gpt embed model. Each of the variants disabled a component or swapped out the component for an alternative one to see the change in performance.

The No_Cross variant disabled the cross-entropy loss in the training loop, and the No_SVD model disabled the SVD embedding as the dual embedding of the node. From figure 5.2, and table 8, we can see that the average MAE and RMSE are not significantly different,

however, the variance is slightly larger, meaning that with the help of those two components, our model not only provides better performance but also a more stable and reliable outcome.

Finally, with gpt embed mode, we swap the BERT embedding to a GPT based text embedding, the result shows it's less stable with higher variance of output, also the accuracy of the model decreased, which is an evidence of using a BERT bidirectional model can better extract the semantic meaning of the review and help our model to analyse the data.

5.3 Computational Efficiency Analysis

This section explores and analysis the practical computational resources for training the model, as we analysed the time complexity on section 3.7.2, we can expect most of the time on and resource will be spent on data preprocessing since it grows linearly respect to the number of user and items.

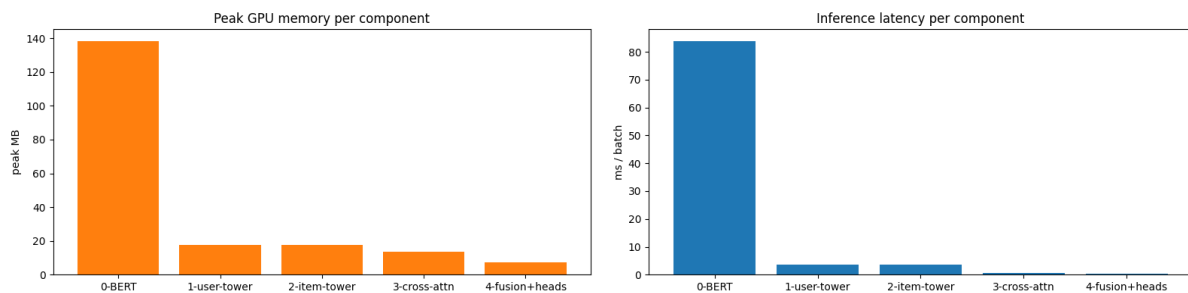


Figure 5.3 the GPU memory usage and the average time required to process a batch in each component of our architecture

From Figure 5.3, we can see that the BERT embedding required most of the space of the time to process, with almost 140MB per batch (in our dataset we have 2048 batches) and over 80 ms to process each batch. This shows our model could be computationally expensive in terms of GPU memory and processing time, however, noticing other components, user tower, item tower, cross attention and fusion heads, is relatively cheap, with more than 7 times differences to BERT embedding process in GPU memory usage and more than 16 times difference in training time. The result is ideal, since most computational expensive part, data embedding, can be isolated and process in other program with multithread and then cached, our core model will be efficient in the rating prediction task. Furthermore, since the rating prediction architecture is light weighted, it means it could be used in real-life example with different embedding or pre-processed data.

5.4 Convergence and Training Dynamics

This section explores the convergence of our proposed model over the training epochs.

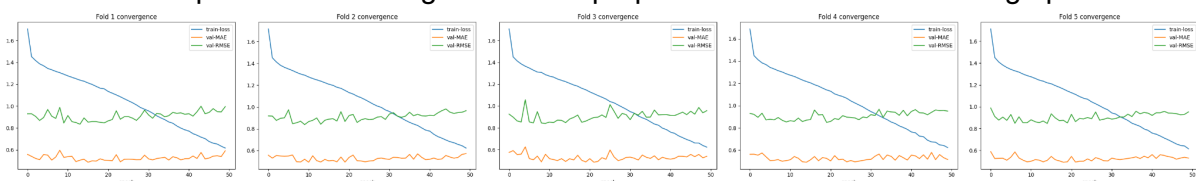


Figure 5.4 convergence of the model on Amazon Beauty dataset, in each of the 5-fold cross validation, showing the training loss (blue line), training RMSE value (green line) and training MAE value (yellow line)

For the purpose of showing the convergence, we extend the training epoch to 50 to show over time what will the loss value and result is in both MAE and RMSE metrics.

We can see that overtime, the composite loss function does converge over the training epochs in all 5 folds, which proves the convergence of our model, however, if we look carefully on the actual performance of the model, most fold will have its lowest MAE and RMSE value in between 15 to 20 epochs, after that the result rises slightly, this shows that we choose the best the best epochs for this dataset, as over training our model architecture might lead to a overfitting problem.

6. Discussion

The experimental results demonstrate that our model substantially improves recommendation rating prediction accuracy through its innovative hybrid architecture. Across two different benchmark domains, Amazon Beauty and Amazon Books, our model delivered state-of-the-art performance in rating production, reducing MAE by approximately 19% relative to strong baseline methods. Notably, BEST-REC achieved the lowest prediction error (both MAE and RMSE) on the sparsely reviewed All_Beauty dataset, and it attained the best MAE with a competitive RMSE value on the larger Books dataset. This indicates that the model is highly effective, especially in sparse settings where the traditional approach struggles. In the Books domain, several baselines, including WDL, RGCL, P5 and Exp3RT, reached accuracy comparable to our model. However, in the Beauty domain, our model significantly outperformed all baselines. This might be because all models used in the Books domain incorporate the qualitative text data in some way, underlining that the integration of review text for sentiment analysis provides a clear advantage over using only quantitative data in cold-start or sparse scenarios. Furthermore, the ablation study also shows strong evidence of stronger performance when incorporating both qualitative and quantitative data. These results confirm that BEST-Rec effectively addresses the core challenges of data sparsity and cold-start recommendation by leveraging heterogeneous information and deep feature interaction mechanisms.

Unlike prior recommenders that treat each data modality in isolation, BEST-Rec combines multiple information sources in a unified model. Most importantly, a bi-directional cross-attention mechanism aligns user and item textual representations during feature learning, rather than only after independent encoding. This design allows fine-grained alignment of user preferences with item attributes in the latent space, capturing subtle matching signals that earlier architectures might miss. The strong performance improvements we observe can be attributed to this synergistic fusion of heterogeneous data. The transformer architecture provides a relational context and deep semantic understanding of unstructured text. Furthermore, the cross-attention tightly couples user-item features to reveal relevant interactions.

An ablation study was conducted to quantify the contribution of each architectural component and the affect of qualitative and quantitative data. the results were uniformly positive - each component substantially boosts the model's accuracy. Key findings from the abltion includes:

1. **Cross-Attention Fusion:** Removing the cross-attention between user and item textual embeddings led to a drop in accuracy, confirming that the cross-attention module contributes critically to model effectiveness. By allowing the model to dynamically emphasise aspects of an item most relevant to a given user, the cross-attention mechanism captures individualised preference alignments that would be lost with a simple concatenation or late fusion approach. This fine-grained interaction was a core hypothesis of our design, and the ablation results validate that hypothesis: features learned with cross-attention yield better predictions than features learned independently.
2. **Dual-source item embedding (BERT + SVD):** we found that both the BERT-based textual embedding and the SVD-based latent factor are indispensable and complementary. In ablation variants where one of these components was removed, performance degraded, indicating that each captures information that the other cannot. The BERT encoder provides a rich semantic representation of items by extracting meaning from textual reviews and descriptions, while the low-rank SVD embedding injects collaborative filtering signals in a compact form. Together they form a more holistic item profile: the BERT embedding captures nuanced item attributes and user opinions, and the SVD embedding summarises global preference patterns while denoising noise in interaction data. This dual representation yielded clearly better results than either source alone. Furthermore, we found that BERT-based textual embedding is critical by swapping this component out to a GPT-based textual embedding. We can notice that without BERT-based embedding, not only does the performance drop significantly, but also the variance of the 5 fold CV increased, meaning more unstable outcomes and prediction by using the GPT-based embedding.

In addition to architectural ablations, we also examined the importance of data modalities by training on selectively truncated input data. The findings reinforce our design choices. Excluding textual reviews (the "no-text" variant) caused a dramatic performance degradation - the average MAE spiked from around 0.52 to 0.99, and RMSE from 0.89 to 1.42 on the beauty dataset. This nearly doubling error highlights that qualitative textual data are indispensable for accurate rating prediction, and it confirms that our model effectively exploits review semantics. Conversely, excluding the structured numeric metadata (the "no-numeric" variant, using only text and collaborative data) had only a mild effect on performance - MAE rose slightly by 0.01 and RMSE rose by 0.01. The model still performed nearly as well with just reviews and latent factors, indicating that textual and collaborative signals carry the bulk of the predictive signal, while metadata such as price, rating count and others provides a useful but comparatively smaller boost. This result suggests that our model's strength indeed lies in the textual and transformer-based components, aligning with our focus on those for tackling sparsity. It also implies that in domains where metadata is scarce or unreliable, our model would still perform strongly by relying on the other information source.

When comparing our model's strengths to baseline models, several observations emerge. Simpler methods like MF or KNN were clearly outperformed - for instance, KNN had the worst accuracy, suffering heavily on the sparse data due to its lack of learned representations and inability to use auxiliary information. This underscores the value of learned embedding approaches and incorporating rich side information in modern

recommenders. Compared to recent deep baselines, our model's advantage are most pronounced under data sparsity. On All_Beauty, our model's MAE/RMSE values were markedly better than all alternatives, with relative RMSE improvements exceeding 18% of the current best model. The beauty dataset has extremely sparse interactions, so methods leveraging only limited signals struggle, whereas our model excels by drawing on multiple sources to make up for missing data.

Our model combines the strengths of diverse recommendation paradigms, content-based filtering, collaborative filtering and graph-based learning, which allowed it to outperform or match the state-of-the-art across different conditions. Even in cases where other models achieved similar accuracy, BEST-Rec design offers a practical advantage: it is tailored to explicitly handle cold-start issues and could be more data-efficient. The consistent gains across both datasets and especially for new users/items validate the effectiveness of our approach in addressing the longstanding challenges of cold-start and sparse-data recommendations.

Future Work

Future research directions include: (1) lightweight model variants for large-scale deployment; (2) integration with top-N recommendation pipelines through candidate re-ranking strategies; (3) domain adaptation techniques for cross-domain generalization; and (4) incorporation of multimodal signals beyond text for enhanced cold-start handling.

7. Limitations & future works

Despite its success and strong performance, and innovative design, BEST-Rec has several limitations that open avenues for future improvement. We outline key challenges and propose directions for future work below:

1. **Scalability and Efficiency:** One limitation of the current model is its complexity and computational cost. Our model combines transformer-based text encoders, cross-attention and a transformer model to do rating prediction, which can be resource-intensive for large-scale deployments. Training a larger dataset may not scale linearly, increase the time and space needed for training significantly. Future work should explore lightweight variants and model compression techniques to improve scalability.
2. **Integration with Top-N Recommendation Pipelines:** In its current form, BEST-Rec is designed for predicting explicit ratings. Deploying it in a real-world recommender system would require integrating with the typical top-N recommendation pipeline, where the goal is to retrieve and rank a list of the most relevant items for each user. A direct application of our model for top-N ranking could be challenging because scoring every item for every user with a heavy model is computationally prohibitive. To address this, future research should explore a two-stage approach: using BEST-Rec as a re-ranking model on a small set of candidate items. By using a smaller model to retrieve a shortlist of potentially relevant items for user, then use our model to score and rank the shortlist using its detailed features could be a possible idea.

3. **Generality and domain adaptability:** Our evaluation was limited to two product domains, Beauty and Books, with explicit rating prediction as the task. It remains to be seen how well our model generalises to other domains or other types of user feedback, as different groups of people in different domains might provide feedback differently. The model currently relies heavily on rich review text; in domains where such text is sparse, or in languages not well handled by our pretrained language model, performance might suffer. Additionally, the cold start problem is mitigated but not completely resolved: if a new user has provided no review at all, our model has little information to go on. In future work, when data on user profiles or social network data is accessible, a supplementary model could be trained and used to handle extreme cold-start cases. Another future direction is to apply transfer learning or domain adaptation techniques so it can better generalise its learned representations to a new domain with minimal fine-tuning.

In summary, addressing these limitations involves making the model more efficient, more general and more aligned with real-world recommendation settings. By tackling these challenges, we hope to make BEST-Rec not only an academic contribution but also a practical solution for modern recommendation systems.

8. Conclusion

In this thesis, we proposed BEST-Rec, a novel hybrid recommender system designed to tackle the challenges of cold-start users/items and extreme data sparsity in recommender systems. Our approach unified heterogeneous information sources- including unstructured textual reviews, structured item metadata, and latent collaborative factors - within a single end-to-end model. We introduced an attention-based feature fusion mechanism, which uses bidirectional cross-attention between user and item representations, to enable fine-grained interactions during representation learning, rather than only merging features after independent encodings. This addresses a key research gap by allowing the model to align user preferences with item characteristics at a detailed level, capturing subtle signals that traditional architectures might miss.

Each item in our model is represented by dual embeddings (BERT-derived text embedding and SVD-derived collaborative embedding), and each user is represented via their review-based transformer encoding, all of which are enriched through an attention mechanism. Training is guided by a composite objective that combines L1 regression loss for accuracy and cross-entropy for classification as the regularisation and guideline for the model. Together, these innovations form a comprehensive solution to the stated problems: the model can ingest multiple modalities in one architecture and allow direct user-item interaction modelling during training, fulfilling needs outlined in our research questions.

Through extensive experiments on two real-world datasets, Amazon Beauty and Books, we demonstrated the effectiveness of BEST-Rec. The model achieved state-of-the-art rating prediction accuracy, outperforming most baseline models on MAE and RMSE metrics. Crucially, the gains were largest in scenarios with very sparse data and for cold-start entries. Our approach yielded significantly lower error for users with fewer reviews and items with

scant interaction history, where conventional methods often fail. We showed that each component of our architecture contributes to this success: the cross-attention fusion improved the alignment of user-item features, the dual source embedding provides complementary strengths from content and collaborative data, and the cross-entropy loss enhanced generalisation of the model and led the direction of training to improve accuracy. The ablation studies and comparative evaluations validate that our design choices directly address the challenges of sparsity and cold-start. In particular, by leveraging review text, our model can make accurate predictions even when data is limited, thereby answering the core research question of how to effectively utilise side information to alleviate sparsity.

Overall, the research findings confirm that integrating heterogeneous relational context with advanced transformer architectures offers a principled remedy for long-standing recommendation problems. This work not only provides a high-performing model for rating prediction, but also offers insight into building more resilient recommender systems: it underscores the value of combining content-based and collaborative knowledge, and of allowing rich interactions between user preferences and item attributes. In conclusion, BEST-Rec represents a significant step forward not only in tackling cold-start and sparse data issues, but furthermore it enhances performance in general rating prediction task in a general setting. It contributes a flexible and powerful framework that can be built upon in future studies, moving us closer to a recommender system that remains accurate and reliable even when data is limited or missing. The approaches developed in this thesis lay a strong foundation for next-generation recommenders that are both data-efficient and capable of leveraging the full spectrum of information available, thereby better serving users with personalised and relevant recommendations.

- [1] Najafabadi, M. K., Chen, R. A., Rezazadeh, J., Beheshti, A., & Shabani, N. (2025). From theory to practice: The evolution and comparative analysis of homogeneous vs. heterogeneous Graph Neural Networks in recommender systems. *Neurocomputing*, 624, 129446.
- [2] Hasan, E., Rahman, M., Ding, C., Huang, J., & Raza, S. (2024). Based recommender systems: a survey of approaches, challenges and future perspectives. *ACM Computing Surveys*.
- [3] Najafabadi, M. K. (2024). Sentiment analysis incorporating convolutional neural network into hidden Markov model. *Computational Intelligence*, 40(2), e12633.
- [4] Nguyen, T. H., & Tran, T. D. (2024). Product recommendation for online sales systems based on transaction sessions. *CTU Journal of Innovation and Sustainable Development*, 16(Special issue: ISDS), 17-24.
- [5] Liu, J., Liu, C., Zhou, P., Lv, R., Zhou, K., & Zhang, Y. (2023). Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149*.
- [6] Li, M., Que, W., Geng, Z., Li, M., Kou, Z., Chen, J., & Zhang, B. (2023). Cold-start item recommendation for representation learning based on heterogeneous information networks with fusion side information. *Future Generation Computer Systems*, 149, 227-239.
- [7] Liu, Y., Yang, S., Zhang, Y., Miao, C., Nie, Z., & Zhang, J. (2021). Learning hierarchical review graph representations for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 35(1), 658-671.
- [8] Najafabadi, M. K., Mohamed, A., & Onn, C. W. (2019). An impact of time and item influencer in collaborative filtering recommendations using graph-based model. *Information Processing & Management*, 56(3), 526-540.
- [9] Shuai, J., Zhang, K., Wu, L., Sun, P., Hong, R., Wang, M., & Li, Y. (2022, July). A review-aware graph contrastive learning framework for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 1283-1293).
- [10] Xu, S., Sun, F., Wu, X., Zhang, W., Zhang, Z., & Wang, S. (2023). A heterogeneous graph neural recommendation model with hierarchical social trust. *Computers and Electrical Engineering*, 111, 108912.
- [11] Najafabadi, M. K., Mahrin, M. N. R., Chuprat, S., & Sarkan, H. M. (2017). Improving the accuracy of collaborative filtering recommendations using clustering and association rules mining on implicit data. *Computers in Human Behavior*, 67, 113-128.
- [12] Li, X., Sun, L., Ling, M., & Peng, Y. (2023). A survey of graph neural network based recommendation in social networks. *Neurocomputing*, 549, 126441.
- [13] Jiarun Fu, Rong Gao, Yonghong Yu, Jia Wu, Jing Li, Donghua Liu, and Zhiwei Ye. 2024. Contrastive graph learning long and short-term interests for poi recommendation. *Expert Systems with Applications*, 238:121931.
- [14] Wang, Y., He, X., Wang, H., Sun, Y., & Wang, X. (2022). Fast Explainable Recommendation Model by Combining Fine-Grained Sentiment in Review Data. *Computational Intelligence and Neuroscience*, 2022(1), 4940401.
- [15] Kharidia, V., Paprunia, D., & Kanikar, P. (2024, October). LightFusionRec: Lightweight Transformers-Based Cross-Domain Recommendation Model. In *2024 First International Conference on Software, Systems and Information Technology (SSITCON)* (pp. 1-7). IEEE.

- [16] Kim, J., Kim, H., Cho, H., Kang, S., Chang, B., Yeo, J., & Lee, D. Review-driven personalized preference reasoning with large language models for recommendation. CoRR, abs/2408.06276, 2024. doi: 10.48550. *arXiv preprint ARXIV.2408.06276*.
- [17] Chen, L., Wang, X., & Zhang, B. (2020). A survey on deep learning for recommender systems. *Neurocomputing*, 396, 411-427.
- [18] Wu, L., Zheng, Z., Qiu, Z., Wang, H., Gu, H., Shen, T., ... & Chen, E. (2024). A survey on large language models for recommendation. *World Wide Web*, 27(5), 60.
- [19] Liu, P., Zhang, L., & Gulla, J. A. (2023). Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems. *Transactions of the Association for Computational Linguistics*, 11, 1553-1571.
- [20] Gao, C., Wang, X., He, X., & Li, Y. (2022, February). Graph neural networks for recommender system. In *Proceedings of the fifteenth ACM international conference on web search and data mining* (pp. 1623-1625).
- [21] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30-37.
- [22] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 173 – 182.
- [23] Liu, Y., Yang, S., Zhang, Y., Miao, C., Nie, Z., & Zhang, J. (2021). Learning hierarchical review graph representations for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 35(1), 658-671.
- [24] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017, April). Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web* (pp. 173-182).
- [25] Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 73–105). Boston, MA: Springer US.
- [26] Noorian, A., Harounabadi, A., & Hazratifard, M. (2024). A sequential neural recommendation system exploiting BERT and LSTM on social media posts. *Complex & Intelligent Systems*, 10(1), 721-744.
- [27] He, R., & McAuley, J. (2016, April). Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *proceedings of the 25th international conference on world wide web* (pp. 507-517).
- [28] Zhang, Y. (2022). An introduction to matrix factorization and factorization machines in recommendation system, and beyond. *arXiv preprint arXiv:2203.11026*.
- [29] Li, C., Xu, L., Yan, M., & Lei, Y. (2020). TagDC: A tag recommendation method for software information sites with a combination of deep learning and collaborative filtering. *Journal of Systems and Software*, 170, 110783.
- [30] Shenbin, I., Alekseev, A., Tutubalina, E., Malykh, V., & Nikolenko, S. I. (2020). Recvae: A new variational autoencoder for top-n recommendations with implicit feedback. In *Proceedings of the 13th international conference on web search and data mining* (pp. 528-536).
- [31] Chen, H., Lin, Y., Pan, M., Wang, L., Yeh, C. C. M., Li, X., & Yang, H. (2022, September). Denoising self-attentive sequential recommendation. In *Proceedings of the 16th ACM conference on recommender systems* (pp. 92-101).
- [32] Cui, Y., Yu, H., Guo, X., Cao, H., & Wang, L. (2024). RAKCR: Reviews sentiment-aware based knowledge graph convolutional networks for Personalized Recommendation. *Expert Systems With Applications*, 248, 123403.

- [33] Do, P. M. T., Zhang, Q., Zhang, G., & Lu, J. (2023). meta-GRS: A graph neural network for cross-domain recommender system via meta-learning. *Procedia Computer Science*, 225, 2536-2545.
- [34] Ngo, H., & Nguyen, D. Q. (2024). Recgpt: Generative pre-training for text-based recommendation. *arXiv preprint arXiv:2405.12715*.
- [35]
- [36]
- [37]
- [38]